阶段一交付完成 | Web 端已部署上线 | 16/16 页面实测可达

AI 厨娘

智能烹饪助手 · 项目答辩

以 LangGraph 多智能体编排为核心，串联「食材输入 → 智能推荐 → 分步引导 → 营养分析 → 库存管理 → 会员服务」全链路， 并以纯规则风控引擎守住食品安全底线。三端 74 页、前端 47,736 行代码、后端 121 个接口。

PRESENTED BY

肖裔雄

DATE

2026.10.14

STATUS

已上线

目录 CONTENTS

四个部分，回答三个问题：做了什么、怎么做的、学到了什么

25 PAGES · 4 CHAPTERS
2026.10.14

01 **项目概述与技术架构** — 7 页
项目背景与产品定位 · 协作模式与交付物 · 交付成果核心指标 · 五层系统架构 · 技术栈选型 · 数据与语料资产

02 **重点模块与精彩代码** — 10 页
智能菜谱推荐 · **LangGraph 主图拓扑** · 路由机制与两张子图 · 图结构运行期校验与自检发现 · 食品安全风控 · 其余核心能力速览 · 两段关键代码 · 真实界面演示

03 **问题与解决方案** — 3 页
六类问题 × 方案 × 验证结果总览 · 四个典型案例深剖（检索缺口 / 构建 OOM / 镜像加速失效 / 语音静默失效）

04 **项目完成后的收获** — 2 页
技术深度 · 工程能力 · 产品思维 · 协作规范 · 风险意识 —— 从「能跑」到「敢上线」的五项成长

本次答辩重点：

- 三大核心模块的实现路径
- LangGraph 图结构与编排设计
- 问题排查与工程取舍

CHAPTER 01

01

项目概述与技术架构

先回答「这是什么项目、交付了多少、它长什么样」—— 从产品定位与协作模式，到五层系统架构与技术栈选型，建立全局坐标。

74 页

三端页面总数（Web 16 / 小程序 42 / 后台 16）

47,736 行

前端代码量（三端合计）

121 个

后端接口端点（118 个 py 模块）

项目背景与产品定位

面向家庭做饭人群的智能烹饪助手 · 把「今天吃什么」的决策成本降到最低

1 真实场景里的四个痛点

家庭做饭的困难并不在「不会做菜」，而在决策与核对环节。围绕这一判断，项目立项时明确了四个必须解决的现实问题：

① **食材有了，不知道做什么。**冰箱里剩三样食材，用户需要在多个菜谱站之间反复比对，最后往往还是点外卖。

② **菜谱不适配个人条件。**网上菜谱不考虑厨艺水平与家里的厨具设备，新手照着做经常卡在第一步。

③ **过敏原与忌口靠人记。**家里有过敏成员时，用户只能逐条阅读配料表自行判断，一旦疏漏后果严重。

④ **做完不知道营养是否达标。**缺少可累积、可回顾的饮食数据，控糖、健身人群无从调整。

2 产品定位

一句话概括：**面向家庭做饭人群的智能烹饪助手**，覆盖「食材 → 推荐 → 分步引导 → 营养 → 库存 → 会员」全链路， 把分散在多个 App 里的动作收敛到一条主路径上。

更关键的一点是定位差异：本产品不把食品安全交给用户「自己注意」，而是由系统在**检索与排序之间强制拦截**—— 这是它区别于普通菜谱类应用的核心。

家庭做饭人群 | 有过敏成员的家庭 | 厨房新手 | 控糖 / 健身人群

3 核心形态与技术主张

- **多智能体编排：**用 LangGraph 把意图识别、推荐、烹饪引导、营养、库存、创新菜等能力组织成一张可观测、可扩展的图，而不是一堆互相调用的函数。
- **风控独立且不调大模型：**食品安全引擎是纯规则实现，确定性、可审计、毫秒级响应，且从结构上没有 Prompt 注入面。
- **推荐全程无 LLM：**推荐主路径由「三路召回 + 布尔硬过滤 + 多因子加权」构成，结果可解释、可复现、可测试。
- **三端同源：**Web 演示端、微信小程序、管理后台共享同一套接口与业务逻辑，逻辑层沉淀在共享包中复用。

协作模式与交付物

甲乙方外包 · 我方为乙方（开发方）· 交付物 B-D1 ~ B-D8 八类

四条协作约定

01 文档权限分层

01 需求基线乙方只能评论不能修改；06 合规区只读；07 项目管理区双方共编。权限边界清晰，避免需求被单方面改动。

02 冲突以需求文档为准

乙方产出的 PRD 与需求文档发生冲突时，一律以甲方需求文档为准。PRD 不重写需求，只做交互细化与章节引用。

03 变更必须走变更单

任何需求调整都必须提报 `CR-YYYYMMDD-NN` 变更单，**口头变更一律无效**，保证需求基线可追溯。

04 术语口径统一

US-xx 用户故事 | FR-x.x 功能需求 | NFR-x 非功能 | RL-x 合规红线 | Q-xx 待确认项。红线与变更单严格区分，不混用。

八类交付物清单

B-D1 PRD 产品需求	**B-D2** UI 设计稿	**B-D3** 架构设计	**B-D4** 测试方案
B-D5 源码交付	**B-D6** 部署运维接口	**B-D7** 测试报告	**B-D8** 开源组件清单

交付节奏

阶段一 · 7 个工作日：Web 可演示原型

合同基准为 6 页（W-01 ~ W-06），实际交付 **16 页**，超额 167%；后端接口、风控引擎、推荐子图同步完成，并提前把 RL-14 敏感信息单独同意落地。

阶段二 · 16 周：微信小程序 + 管理后台

代码层已完成 42 页小程序与 16 页后台，并全部注册进 pages.json；剩余工作集中在备案、支付签约、真机联调三件事上。

交付成果核心指标

全部数据来自本地实测与线上实探，可用附录命令独立复现

WEIGHTED COMPLETION

≈ 88%

74

交付页面（三端）

Web 16 + 微信小程序 42 + 管理后台 16，全部完成注册与路由配置

47,736

前端代码行数

Web 18,486 + 后台 3,375 + 小程序 25,875，逻辑层通过共享包复用

121

后端接口端点

118 个 py 模块、3 个数据库迁移、46 个测试文件，覆盖推荐与风控全链路

319

后端单测通过

319 passed / 1 skipped；前端另有 236 + 45 条用例，43 条 E2E 全闭环

三端实现明细

Web 演示端 `web-stage1` — 16 页 · 18,486 行

Vue 3 + Vite + TS + Pinia + Element Plus；含 W-06 Agent 调度可视化面板等专属交付页

微信小程序 `web-mini` — 42 页 · 25,875 行

uni-app + Vue 3 + Pinia，六个分包：主包 / recommend / cooking / nutrition / member / innovation

管理后台 `admin-web` — 16 页 · 3,375 行

风控规则、违禁词、创新巡检、开关管理（远程配置）、操作日志、调度日志等运营侧能力

质量与上线

页面实测可达	16 / 16
控制台 Error	0
安全红线	12 条全绿
E2E 用例	43 条 · 缺陷全闭环

生产环境实探

82.156.216.134:8005

用户端 / 后台 / healthz 全部 200, DB up

系统架构总览

五层分层架构 · 端到端单机 Docker Compose 部署 · 四服务（web / api / db / redis）

① 用户侧
CLIENTS

Web 演示端（16 页） · 微信小程序（42 页） · 管理后台（16 页）

三端共用同一套 REST 接口；Web 与后台使用 Vue 3 + Element Plus，小程序使用 uni-app，业务逻辑在共享包中复用

② 接入层
NGINX

Nginx 容器 80 端口映射到宿主机 **8005**：静态资源托管 + /v1、/api、/admin/v1 反向代理

已启用 gzip 与静态资源缓存；TTS 流式接口单独配置 proxy_buffering off，保证边合成边播

③ 应用层
FASTAPI

Python 3.11 + FastAPI，**121 个接口端点**，按业务域分组：auth / recommend / cooking / nutrition / inventory / member / innovation / speech / vision / profile / home / trace

应用层依赖 Provider 抽象：LLM、视觉模型、语音、Embedding 全部可切换，缺省回退 mock 不影响启动

④ 编排与风控
LANGGRAPH

LangGraph 编排中枢：22 个节点（主图 13 + 子图 9），全图仅 1 个 LLM 节点（supervisor，仅做意图识别）

风控引擎为纯规则实现（27 条规则），位于主图入口与出口两侧，形成输入 / 输出双闸门；命中即控制流短路

⑤ 数据层
POSTGRES

PostgreSQL 16 + pgvector（单库 aicook，28–32 张表，3 个 Alembic 迁移）＋ Redis 7 缓存预留

语料 373 篇菜谱 md + 371 张封面图；创新菜写入用户私有向量分区，与公共知识库物理隔离

写进设计里的六条硬约束

这些不是「建议」，而是代码结构中强制的规则

- ✓ 风控引擎**不调用大模型**，保持确定性
- ✓ 一票否决 = **控制流短路**，不是标记后丢弃
- ✓ 过敏原走**布尔硬过滤**，绝不降权
- ✓ **双重风控**：输入侧与输出侧都不可省
- ✓ 风控异常或超时 → **保守模式**，绝不直连模型
- ✓ need_user_confirm 恒为 true，识别结果必须用户确认

另有 4 个模块级远程开关强制交付，可通过后台热更新，无需重新发版。

技术栈选型

一个核心服务 + 三类技术支撑 · 外部 AI 能力全部 Provider 化，可 mock 与真实一键切换

前端 · 三端同源

Web / 后台： Vue 3 + Vite + TypeScript + Pinia + Element Plus

小程序： uni-app + Vue 3 + Pinia，六个分包组织 42 页

共享层： packages/shared 沉淀请求、状态与业务逻辑

选型理由：一套 Vue 心智模型覆盖三端，降低多端维护成本

后端 · 单体服务

框架： Python 3.11 + FastAPI（异步、自动 OpenAPI 文档）

数据访问： SQLAlchemy 2.0 + Pydantic v2 校验

规模： 118 个 py 模块、121 个端点、46 个测试文件

选型理由：Pydantic 让接口契约与文档同源，前端联调成本大幅下降

数据 · 单库起步

主库： PostgreSQL 16，单库 aicook，28–32 张表

向量： pgvector 扩展，与业务数据同库事务一致

缓存： Redis 7 已部署并预留，阶段二接入

选型理由：阶段一不引入额外中间件，规模化后再迁 Milvus

AI 能力 · 全部可换

意图识别： LLM（全图唯一 LLM 节点）

拍照识别： VLM Provider，真实模型与 mock 可切

语音： ASR / TTS（qwen3-asr-flash / qwen3-tts-flash）

嵌入： 当前为哈希伪向量，可切语义模型

选型理由：Provider 抽象让 AI 能力可替换、可降级、可测试

部署与运行方式

容器编排

Docker Compose 单机四服务；`api` / `db` / `redis` 端口不对外发布，仅 nginx 暴露 8005，减少攻击面。

构建策略

前端镜像峰值内存超 1.5GB，采用「本地构建 → save → scp → load」，规避 2C2G 服务器 OOM。

资源守卫

Compose 设定保守上限：web 192M、api 768M、db 512M、redis 128M，防止单服务拖垮整机。

部署脚本

deploy.sh 固化 8 个步骤，含语料硬校验、占位符拒绝、端口断言与健康检查轮询，可重复执行。

数据与语料资产

推荐质量的地基不是模型，而是语料本身 —— 373 篇菜谱 + 371 张封面 + 27 条风控规则

菜谱语料库

373
篇菜谱 md

371
张封面图

来源与授权：HowToCook 开源菜谱集（Unlicense 公有领域，v1.6.0），版权风险可控。

组织方式：按 10 个分类目录存放，带图菜谱采用「同目录同名 .md + .jpg」约定，解析器递归扫描并剥离图片行。

运维支持：配套 7 个语料工具脚本，可批量迁移缺图、分发封面、规范化命名、校验结构。

⚠ 一条容易被误读的口径

语料导入时**不落库图片行**，封面字段默认留空——也就是说「补了图」不等于「产品里有图」， 卡片上真正显示图片还需要独立的图片链路改造。这条口径在验收文档中单独标注，避免验收时产生分歧。

风控规则库 · 27 条

主库 24 条 + 创新菜专项 3 条

FS- 食品安全高风险	发芽土豆、久泡木耳、未熟四季豆等	11
MR- 医疗拒答	诊断、治疗类诉求一律拒答	4
OB- 输出禁用表述	绝对化用语、安全承诺、违禁词	5
SP- 特殊人群	孕产妇、婴幼儿等敏感人群提示	2
PS- 伪科学 / 食物相克	仅提示不阻断，避免误伤正常提问	2
INNO- 创新菜专项	医疗 / 输出禁用 / 伪科学，输入输出双侧	3

合规待办与红线

语料文本清洗：上游语料部分描述带功效与医疗宣称表述，需按 RL-7 / RL-11 做专项清洗后才能上架。

备案类资质：生成式 AI 备案、小程序备案、类目资质、虚拟支付签约均未启动，是全项目唯一外部不可压缩的关键路径。

已提前落地：RL-14 敏感个人信息单独同意已完成，准入与免责声明在首屏强制拦截。

阶段豁免说明

阶段一产物不对外提供服务，因此备案与专家签字在合同层面豁免；但阶段二小程序必须全部补齐， 且备案周期不受开发进度影响 —— 这也是我们在阶段一收尾时就把资质办理清单列入 P0 的原因。

CHAPTER 02

02

重点模块与精彩代码

本部分回答「到底怎么做的」：三个核心模块的真实实现路径，其中包含**编译后导出**的完整 LangGraph 图结构、路由机制与两张子图，以及两段最能体现设计取舍的代码。

13
主图节点数（编译后导出的运行期拓扑）

24
主图边数，其中 13 条为显式白名单条件边

3
张 StateGraph：1 主图 + 推荐子图 / 创新子图

18
图结构单测 passed，拓扑红线全部通过

重点模块① 智能菜谱推荐

第一主路径 · 推荐子图 5 节点 · **全程不调用大模型**，确定性、可解释、可审计

模块完成度 98%　0 次 LLM 调用

`assemble_context` **① 上下文归一** — 食材名 / 别名 → ingredient_id

→ `recipe_retrieve` **② 三路混合召回** — 覆盖率 / 文本 / 向量 · ≤200

→ `hard_filter` **③ 布尔硬过滤** — SQL WHERE 四维布尔运算

→ `recipe_rank` **④ 7 因子加权** — 输出 rank_reasons → Top 8　（与⑤并行）

⇉ `node_nutrition` **⑤ 营养估算** — 按过滤输出序取前 3 道

并行语义： ④ 与 ⑤ 由 `hard_filter` 同时 fan-out，在**同一 superstep** 内并行执行，两条分支都连 END，**全部结束后**才返回主图进入 compose —— 因此 ⑤ 无法依赖 ④ 的输出，取的是过滤后的前 3 道。

召回：三路混合与生效条件

① 覆盖率召回 恒生效

按命中食材数量降序排列，是「有什么做什么」这一主场景的基线能力。

② 文本召回 需携带文本

菜名模糊匹配，并从菜谱正文中解析食材提及，补足「按名字找菜」的场景。

③ 向量召回 需 query_text

用 pgvector 计算相似度取 Top-20，捕捉字面不同但语义相近的菜谱。

三路结果取并集去重，再按固定优先级融合。**诚实说明：**纯手选路径不携带文本，因此不触发向量召回，阶段一实际只用到两路。

过滤：四个维度命中即剔除

过敏原
落到数据库布尔运算，绝不降权

忌口
素食、清真、不吃辣等直接排除

难度
不高于用户自评的厨艺能力上限

设备
按家庭厨具做差集，基础工具豁免

四个维度各自记录 `filtered_by` 命中计数，前端可逐条重放「这道菜为什么被过滤」。

口味上的「不喜欢」走的是降权（−0.5）而不是过滤 —— 不喜欢不等于不能吃，两套语义刻意区分。

排序函数的输入集只能是过滤节点的输出集 —— **代码结构本身杜绝「打分后放回」**。

排序：7 因子与结果契约

权重存于远程配置，可热调而不发版

coverage	expiring	taste	goal
.30 生效	.15 未启用	.15 未启用	.15 未启用

difficulty	history	popularity	
.10 生效	.10 未启用	.05 未启用	无效权重 摊给 coverage

结果分档：按缺失食材数分为「全齐 / 缺 1-2 道 / 缺 3 道以上」三档，缺失过多整条跳过。

只取前 8 条：不足时如实提示「现有食材组合有限，宁缺毋滥」，不填充低分菜谱。

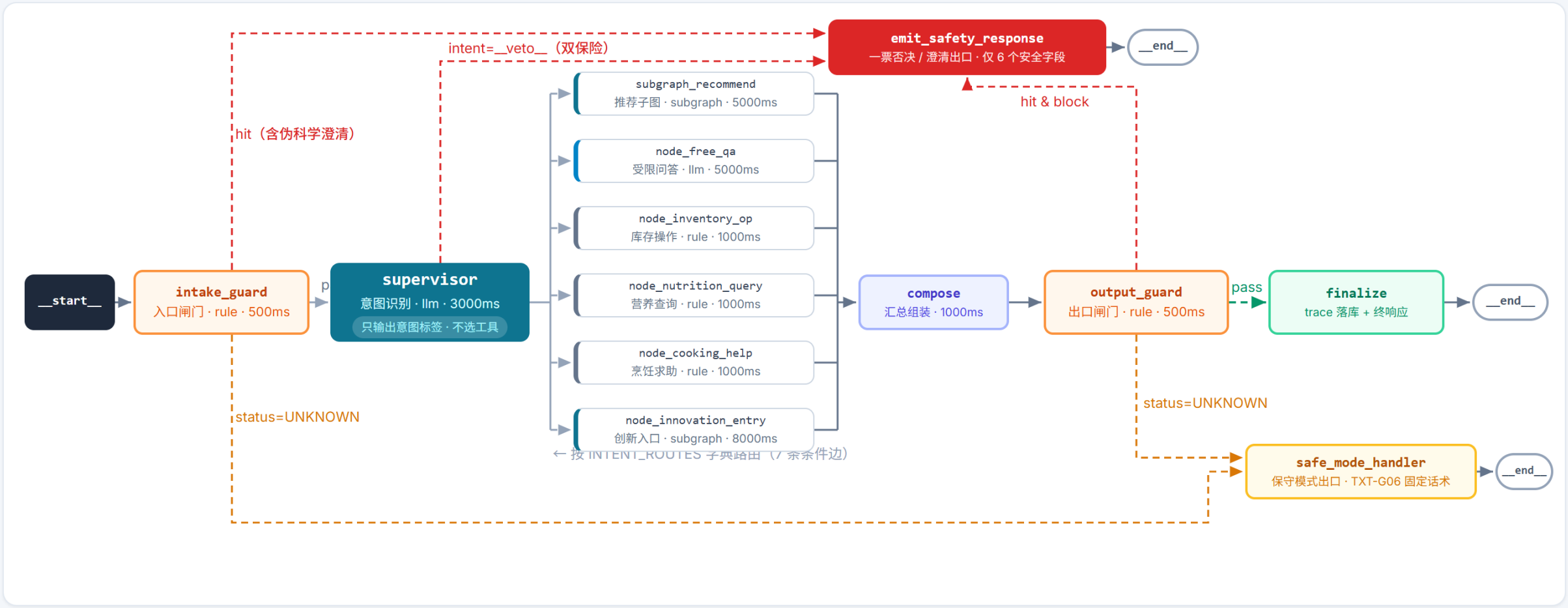
可解释：卡片提供「为什么推荐给你」入口，展开即为 `rank_reasons`。

阶段一真实排序口径：实际仅 coverage 与 difficulty 生效，其余因子接口已就位但权重为 0，随阶段二行为数据积累逐项启用。

LangGraph 主图拓扑（运行期导出）

13 个节点 · 24 条边（含 13 条条件边）· 双侧闸门 + 中枢路由 + 汇聚 + 3 个出口

实线＝普通边　虚线＝条件边　红＝否决短路　琥珀＝保守模式



END 入边仅 3 条
全部来自三个出口，无绕过闸门直达 END 的路径

闸门与 END 无直连
intake_guard 与 END 无直边，否决必经短路出口

条件边白名单闭合
13 条条件边均显式声明目标白名单，越界编译期报错

全节点可达无死点
13 个节点从 START 全部可达，无出度为 0 的死节点

出口闸门不可绕过
output_guard 位于 compose 之后、三出口之前

LangGraph 路由机制与两张子图

3 个路由函数（确定性、零 LLM 参与）· INTENT_ROUTES 10 类意图 · 推荐子图 5 节点 / 创新子图 4 节点

三张 StateGraph 均裸编译 · 无 checkpointer / 无 interrupt

三个路由函数 · 挂载点 / 白名单 / 判定

route_after_intake_guard — 挂在 intake_guard

UNKNOWN→保守模式 | hit→否决出口 | 否则放行 supervisor

route_after_supervisor — 挂在 supervisor

intent==__veto__→否决出口 | 否则查 INTENT_ROUTES，未命中兜底 node_free_qa

route_after_output_guard — 挂在 output_guard

UNKNOWN→保守模式 | hit&&block→否决 | hit&&!block→放行 finalize

关键设计点： LLM 只输出意图标签，「意图 → 节点」映射由代码字典固定，模型无权决定调用哪个工具。

INTENT_ROUTES 全量映射

10 类 · 无遗漏 / 无多余

recommend_by_ingredients	→ recommend	search_recipe	→ recommend
composite_recommend_nutrition	→ recommend	nutrition_query	→ nutrition_query
cooking_help	→ cooking_help	inventory_op	→ inventory_op
innovation	→ innovation_entry	photo_recognize	→ node_free_qa
free_qa	→ node_free_qa	smalltalk_or_out_of_scope	→ node_free_qa

LLM 降级： 捕获 LLMError 或 JSON 解析失败 → `_keyword_fallback` 关键词兜底（覆盖复合意图与创新意图），trace span 标 `degraded`。

推荐子图 subgraph_recommend

5 节点 · 已注册进主图 · recursion_limit 40



并行语义： hard_filter 同时 fan-out 两条边，二者在**同一 superstep** 内并行，都连 END，全部结束后才返回主图 compose。

召回双分支： RETRIEVAL_VECTOR 开关默认 on → pgvector raw SQL / embedding_json 点积，兜底降级不抛错。

并行约束： 营养节点无法依赖排序输出，按过滤输出序取前 3 道估算（与最终 Top 重合度高）。

反馈接线： user_id 存在时加载 dislike 惩罚 −0.5 传入 weighted_rank；无 user_id 排序不变。

创新子图 innovation

4 节点 · 纯线性 · 未注册进主图



非 LangGraph 子图： 它不是主图的子图节点，而是被命令式调用 —— entry 节点 → run_innovation_precheck() + run_innovation_generate() → invoke 子图；同一组函数也被 API 层直接调用，共 **3 个调用入口**。

precheck 红线： 食材逐个过风控扫描（避免安全食材拼接误伤组合规则）→ 过敏原交集非空直接拒绝 → 特殊人群 + 高危食材拒绝 → taboo 命中拒绝；**UNKNOWN / 查表异常一律保守拦截（宁拦不放）**。

generation 约束： 用量 100% 量化（禁「适量 / 少许」）、步骤含时长字段 → 违例重试一次 → 再违例降级本地模板；LLM 硬超时 80.0s；blocked=True 后三个下游节点直接返回 status="skipped"。

父子图状态隔离（关键工程约束 · 源码明确记载「已实测踩坑」）

主图 `CookingState.trace` | 推荐子图 `RecommendSubgraphState.sub_trace` | 创新子图 `InnovationState.sub_trace`

原因： 若子图复用同名 trace 通道，子图会把继承自主图的 span 原样写回，再经主图 `operator.add` 累加 → trace 重复。 **运行期验证：** 一次正常推荐落库 11 行 trace，无重复 span，隔离机制生效。

图结构运行期校验与自检发现

编译并导出拓扑 + 真实调用链路埋点验证 · 图结构单测 18 passed · 同步产出 9 项结构性发现

拓扑红线全部成立 | 18 passed | 9 项发现已登记

finalize — 正常路径出口

触发：正常通过双侧闸门

响应： request_id / results / filtered_by / qa_answer / notice / partial_failures / disclaimer_id / trace_id

取证： 全量 span + 自身 span

emit_safety_response — 否决 / 澄清出口

触发：入口 hit（含伪科学 block=false 的澄清）或出口 hit && block

响应：仅 6 个安全字段，绝不含 results / candidates / nutrition

取证： 为缺失业务节点补 not_executed，output_guard 与 finalize 补 skipped_by_veto，并写 risk_log

safe_mode_handler — 保守模式出口

触发：任一闸门 status=UNKNOWN

响应： safe_mode=true + TXT-G06 固定话术

取证： 业务节点与 output_guard 补 not_executed，并写 risk_log(action="safe_mode")

结构健康度小结

✅ 成立 / ⚠️ 部分 / ❌ 未落地

✅	拓扑红线	全部成立：无绕闸路径、无直连 END、全节点可达、白名单闭合
✅	可测试性	图结构断言已落单测（关键节点 / END 入边 / 否决体纯净），18 passed
✅	状态隔离	子图 sub_trace 隔离有效，一次推荐落库 11 行 trace 无重复
⚠️	可插拔 NFR-4.1	仅节点实现可插拔，新增节点仍需手改 main_graph.py 的边与白名单
❌	超时治理	注册表 timeout_ms 未接入执行，超时与降级矩阵未落地
⚠️	取证完整性	node_innovation_entry 缺 not_executed 补标，否决 trace 取证有缺口
⚠️	trace 一致性	创新链路自生成 trace_id，同一请求被拆成两个 thread_id
⚠️	冗余执行	创新主图路径上 precheck 被重复执行 3 轮
⚠️	文档一致性	设计文档与实现存在多处漂移（子图清单 / checkpointer / 意图类数）

9 项自检发现 · 按修复优先级

均为运行期实测确认

P0 · BUSINESS_NODE_NAMES 追加未生效

对模块自身命名空间重绑定（import 是值拷贝），common 与 guards 的清单都不受影响 → 否决 trace 漏补该节点；改动量极小但直接影响取证链完整性

P1 · 创新路径 precheck 重复执行 3 次

主图节点 → generate 内部 → 子图节点各跑一遍，纯浪费 DB 往返与规则匹配（功能无错，延迟与负载被放大）

P1 · 创新子图 trace_id 断裂

子图自生成 trace_id 与主图不同 → 按 thread_id 做会话级回放会断裂（按 request_id 查询仍可命中）

P2 · 注册表 timeout_ms / forms 从未被消费

全仓库零读取点 → 超时矩阵未强制执行，声明与实际约束能力不一致，易被误读为已实现

P3 · 5 项文档与实现漂移

state.safe_mode 只写不读 · 三图均无 checkpointer / interrupt · 设计文档描述的子图大部分不存在 · 若干 docstring 与实现不一致 · 否决 trace 中包装节点未补标

两个运行期取证案例

一票否决 「木耳泡一晚上明天炒行吗」→ 落库 **15 行** trace：业务节点全为 not_executed，响应键恰为 6 个安全字段，无业务字段泄漏。

伪科学澄清 「豆腐和菠菜一起吃会相克吗」→ 走同一出口但 blocked=false、risk=PS-001 → 返回**科普澄清**而非拦截。

其余核心能力速览

除推荐、风控、编排三条主线外，另有六项能力同样进入阶段一交付范围

15 个模块中 12 个完成度 ⩾ 90%

F 分步烹饪引导

把菜谱转成厨房里真正能照着做的操作流：大字单步显示、进度条、上一步/下一步本地即时切换、并行计时器（5 个预设计时 + 自定义）。

5 态状态机：未开始 → 进行中 → 评分 → 完成，仅进行中可放弃

过敏原前置检查：创建会话即检出冲突，须用户确认才能开工

求助限 3 次/会话：防滥用，退出需二次确认

G 营养分析

单菜谱营养按「每份」口径给出，支持份数缩放重算；另有按日、按餐的汇总接口，为控糖与健身人群提供可累积的饮食数据。

缺失不按 0 计：数据缺失如实标注「未纳入统计」，分母同步排除

手动补录先过闸门：描述文本先经风控校验再落库

避免误导性结论：不给出法定营养声称类表述

H 库存与购物清单

维护家庭在库食材，按临期天数优先排序；临期数据会回流到推荐排序的 expiring 因子，形成「快过期先吃掉」的正向循环。

扣减两段式：先出建议、再确认生效，防误扣

留痕设计：确认扣减时写入变更记录，可回溯

与购物清单联动：缺料自动汇总为采购项

J 会员与支付

六项会员权益口径已对齐；支付回调做了幂等处理，重复请求不会重复入账。识别额度按免费 3 次/日、会员 10 次/日区分。

平台支付抽象层已建：接真实支付只需实现适配器

红线：虚拟支付必须走小程序虚拟支付，禁止跨端引流

红线：iOS 未签约时完全隐藏购买入口

C 用户档案与合规同意

家庭成员、过敏原、厨艺与设备、饮食目标在同一页承载。档案不是普通设置页 —— 它直接驱动推荐硬过滤的四个维度。

过敏原为必填：缺失档案会直接放大食品安全风险

设备白名单校验：避免出现无法执行的菜谱

PIPL 单独同意：RL-14 提前落地并落库留证

♪ 语音交互能力

做菜的双手是湿的、眼睛在看锅，语音是最自然的输入方式。已支持语音转文字、文字转语音、流式边合成边播，以及语音指令意图识别。

工程特点：全走 HTTP 外部服务，无新增依赖、不落盘

配置项均有默认值：缺省回退 mock，不影响启动

硬约束：语音输入依赖 HTTPS，http 下麦克风静默失效

重点模块② 食品安全与过敏原风控

安全底线 · 零容忍 · 27 条规则 · 纯规则引擎不调用大模型

12 条安全红线全绿

模块完成度 97%

四条设计铁律

风控的可靠性不来自规则数量，而来自这四条不可妥协的结构约束

① 不调用大模型

纯规则实现意味着确定性、可审计、毫秒级响应、零 token 成本，而且**不存在 Prompt 注入的攻击面**。安全判定不能是概率问题。

② 一票否决 = 控制流短路

命中即经条件边直达终止节点，**不是「打个标记然后丢掉结果」**。区别在于：短路状态下业务节点根本不执行，也不产生任何业务字段。

③ 双重风控不可省

输入侧拦截用户提问，输出侧拦截模型生成内容。两个闸门面对的风险不同：前者是「不该问的」，后者是「不该说的」，任何一个都不能替代另一个。

④ 降级保守，绝不冒险

风控引擎异常或超时，一律进入保守模式并返回安全话术，**绝不因为「风控挂了」就直接把请求透传给模型**。可用性让位于安全性。

过敏原处理的架构位置 —— 这是全项目最关键的一处设计

检索召回 → **独立布尔过滤节点** → 加权排序

过敏原**不是降权项，而是布尔硬过滤**，独立夹在检索与排序之间，最终落到数据库的 WHERE 条件上。

更关键的是调用关系：排序函数的输入集**只能**是过滤节点的输出集。这意味着即便后续有人想改动逻辑，也无法让一道含过敏原的菜谱进入打分环节 —— 代码结构本身杜绝了这种可能。

与之对照，口味上的 `dislike` 采用**降权（−0.5）**而非过滤：用户不喜欢不等于不能吃，两者语义刻意区分开。

拦截链路 · 四步闭环

① 规则命中

输入或输出文本匹配规则库，记录规则 ID 与版本

② 控制流短路

条件边直达终止节点，跳过全部业务逻辑

③ 前端拦截页

三段式呈现：风险说明 / 正确做法 / 就医引导

④ 可追溯留证

trace 记录规则依据，支持「我认为这是误判」上报

实测证据（全页面测试中真实触发）

在首页输入「发芽土豆能吃吗」并提交 —— 后端风控闸门命中规则 `FS-003`（发芽 / 变青土豆 · 龙葵素）， 控制流立即短路，前端跳转至风控拦截页。

```
命中规则：FS-003 | 类别：食品安全高风险 | 等级：high
规则版本：FS-003@rules-v1.0 | 返回体：不含任何业务字段
页面标注：「本次请求未生成任何内容」
```

精彩代码① 三路召回与布尔硬过滤

过敏原硬过滤为什么必须是布尔运算，而不是「降权」—— 一段代码讲清设计取舍

backend/app/graph/subgraphs/recommend.py　　Python 3.11 | 推荐子图

```python
# 1) 三路混合召回：覆盖率恒生效，文本需带 query，向量需 query_text 非空
candidates = coverage_recall(ctx) | text_recall(ctx)
if use_vector and query_text:
    candidates |= vector_recall(query_text, top_k=20)

# 2) 四维布尔硬过滤：过敏原落到 SQL 布尔运算，绝不降权
rows = session.execute(
    "WHERE NOT (allergen_codes && %s::text[]) AND difficulty <= %s",
    (user_allergens, skill_level))

# 3) 排序输入只能来自 hard_filter 输出集 — 结构上杜绝「打分后放回」
```

● 过敏原：布尔硬过滤，不降权　　dislike：降权 -0.5　　✓ 12 条安全红线全绿

为什么过敏原不能用「降权」

降权只是把分数调低，菜谱仍在候选集里 —— 一旦候选不足、或权重被远程调低，含过敏原的菜谱就可能重新浮上来。 硬过滤则在数据层面直接剔除，**不存在「浮上来」的路径**。

权重可远程热调

排序权重存放在远程配置中，运营侧可在不重新发版的前提下调整因子配比；但**无论如何调整，都改不动过滤节点的布尔条件** —— 可调的是排序，不是安全。

被过滤的菜谱可以「翻案」

过滤时逐项记录 `filtered_by` 计数，前端提供「显示被我过滤掉的菜」开关，用户可查看每道菜被过滤的具体原因。

这个设计的价值在于：如果档案填错导致误过滤，用户能看见原因并及时修正，而不是面对一个「怎么推荐这么少」的黑盒。

Python 3.11　PostgreSQL 16　pgvector　SQL 布尔过滤

精彩代码② 风控短路与节点动态装配

一票否决 = 控制流短路，不是「标记后丢弃」—— 三行代码守住整条安全底线

「短路」和「丢弃」的区别

标记后丢弃意味着业务节点仍然执行了一遍，只是最后不返回结果 —— 中间可能已经调用了模型、写入了数据。 短路则是条件边直接越过全部业务节点，**连执行机会都不给**。

为什么风控不交给大模型

安全判定需要的是**确定性**：同一句话今天判拦截、明天也必须拦截。 规则引擎毫秒级返回、零 token 成本、可逐条审计，而且用户输入无法通过提示词绕过它 —— 因为它根本不读提示词。

动态装配带来的扩展性

节点按注册表装配，新增一个 Agent 只需注册一项，不需要改动图的构建逻辑。 这让「22 个节点」不是一次性写死的结构，而是可持续生长的骨架。

双保险：两层拦截

第一层在图入口拦截用户输入；第二层在处理完成后拦截输出。 即便某个节点被绕过，各节点入口还会再次检查否决标记，命中即以 `skipped_by_veto` 返回。

规则、节点、出口三层防护叠加，才让「12 条安全红线全绿」这个结论站得住。

backend/app/graph/nodes/guards.py | main_graph.py　　纯规则 · 零 LLM

```python
# ① 风控节点：纯规则引擎，不调用大模型
verdict = safety_engine.check(text, stage="intake")
if verdict.overrides_all: # 一票否决
    return {"route": "emit_safety_response"} # 条件边 → 短路

# ② 图装配：节点按注册表动态加入，新增 Agent 不必改装配逻辑
for name, builder in AGENT_REGISTRY.items():
    graph.add_node(name, builder())

# ③ 各节点入口统一检查否决标记，命中即跳过业务逻辑
if state["input_verdict"].overrides_all:
    return {"skipped_by_veto": True} # 不执行任何业务逻辑
```

- 一票否决：控制流短路　　AGENT_REGISTRY：动态装配　　trace：FS-003@rules-v1.0 可回放

功能演示 · 真实界面

浏览器自动化实测 · 真实导航 + 交互操作 + 截图取证

16/16	0	19
页面可达	控制台 Error	截图存证



推荐结果页 · 可解释的推荐

按匹配程度分三档分组，卡片展示耗时、难度、热量、覆盖率与缺失食材；提供「为什么推荐给你」入口与被过滤明细重放开关；匹配不足时如实说明并提示「宁缺毋滥」。



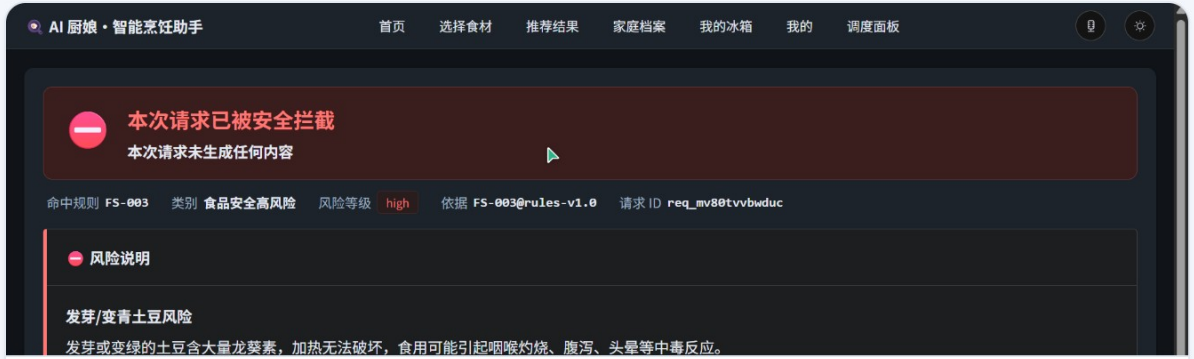
菜谱详情页 · 安全信息前置

顶部常驻致敏原警示条（实测检出大豆）；主料、辅料、调味料三表用量量化并逐行标注致敏原；支持份数缩放重算；页面底部给出营养估算入口与设备要求。



分步烹饪引导 · 厨房场景优先

大字单步显示，进度条标注「第 N 步 / 共 M 步」；上一步下一步本地即时切换无需网络；并行计时器提供 5 个预设计时与自定义；退出需二次确认，支持断点续做。



风控拦截页 · 真实触发

实测输入「发芽土豆能吃吗」命中 FS-003，页面三段式呈现风险说明、正确做法与就医引导，并明确标注「本次请求未生成任何内容」，同时提供「我认为这是误判」的上报入口。

核心链路验证：

- 选食材 → 推荐 → 详情 → 烹饪 全链路跑通
- 风控链路真实触发并正确拦截
- 路由守卫：未选食材 / 无拦截数据 / 未知路径均正确重定向

CHAPTER 03

问题与解决方案

这一部分不复述成绩，只讲踩过的坑：六类问题的「问题 → 方案 → 验证结果」，以及四个从现象追到根因的典型案例。

6

问题类别，覆盖算法、资源、基础设施与浏览器安全模型

3/4

典型案例已闭环，剩余一项已登记为风险并给出触发条件

11

风险登记册条目（R-23 ~ R-33），未闭环项主动披露

问题与解决方案总览

问题 → 方案 → 结果：六类问题全部有落地动作与可验证结论，不做口头承诺

横向逐行对照阅读

① 遇到的问题 六类典型问题	② 解决方案 对应的六个动作	③ 验证结果 可复现的结论
纯手选食材路径不携带文本，向量召回不触发，RAG 只用到一半	补齐三路召回：覆盖率恒生效、文本召回补文本解析、向量召回补条件判定	三缺口已闭环，方案与代码口径对齐并双文档留痕（R-27）
默认嵌入为哈希伪向量，是字面重叠而非语义相似	修复向量升级三缺口：建库命令、环境变量、嵌入空间版本迁移并留痕	剩余限制登记为改进项，随语义嵌入模型升级一并解决
过敏原必须硬过滤，而口味偏好只应降权，两套语义容易混用	过敏原走布尔硬过滤，dislike 走降权 −0.5，语义与实现双重区分	12 条安全红线全绿；被过滤菜谱可在前端逐条重放原因
2C2G 服务器构建前端镜像峰值超 1.5GB，必定 OOM	构建与运行分离：本地 build → save → scp → load，脚本固化进部署流程	生产环境 8005 全端点 200，健康检查返回数据库 up
镜像加速源对写死 registry 前缀的镜像名不生效	先经加速拉取官方镜像，再 docker tag 打上带前缀的别名	部署脚本固化 8 步，含基础镜像准备、语料校验与端口断言
http + 公网 IP 下麦克风静默失效，不报错也不工作	上 HTTPS 或自签证书；同时把该约束登记为风险条目 R-31	端口安全复查与 HTTPS 落地登记为 R-31 / R-33，明确触发条件

关键问题深剖

四个从现象追到根因的案例，按「现象 → 根因 → 解法 → 结论」四段式展开

01 检索召回只用到一半

RAG 链路 · 算法层

已闭环

现象：用户手工勾选三样食材后，推荐结果明显少于预期，且完全不像「语义相近」的菜。

根因：向量召回分支的触发条件写成了 `if use_vector and query_text`，而纯手选路径不产生文本，条件恒为假。

解法：逐项核查三路召回生效条件并补齐缺口，同时修复向量升级遗留的三个工程缺口（构建命令、环境变量、嵌入空间版本迁移）。

结论：三缺口已闭环并双文档留痕；剩余限制（伪向量非语义）明确登记为改进项，不掩盖。

02 2C2G 上构建必然 OOM

部署链路 · 资源约束

已闭环

现象：在服务器上执行前端镜像构建，进程被系统杀死，构建无法完成。

根因：前端 Dockerfile 需要两次依赖安装加一次类型检查，峰值内存超过 1.5GB，而目标机器只有 2GB 内存。

解法：改为「构建与运行分离」—— 本地完成构建后 save 成 tar，scp 传到服务器再 load，服务器只负责运行镜像。

结论：该策略已固化进部署脚本，换机器也能一次跑通；同时给 Compose 加上保守内存上限，避免运行期互相拖垮。

03 镜像加速源「配了但不生效」

基础设施 · 镜像拉取

已闭环

现象：daemon.json 已配置镜像加速地址，拉取部分镜像仍然超时失败。

根因：加速源只对「不带 registry 前缀」的官方镜像名生效；一旦镜像名写死为某个具体 registry 域名，Docker 会直连该域名，绕过加速配置。

解法：先经加速源拉取官方镜像，再用 docker tag 打上带前缀的别名，让构建脚本引用的名字与本地已有镜像对齐。

结论：该处理已写进部署脚本第 1 步，属于「准备基础镜像」的一部分，后续部署不再卡在这一环。

04 语音输入静默失效

浏览器安全模型 · 待办

已登记 R-31

现象：功能代码与接口全部就绪，但在公网 IP + http 环境下点击录音没有任何反应，控制台也不报错。

根因：麦克风采集 API 只在安全上下文（HTTPS 或 localhost）可用；http 页面下浏览器直接禁用权限，属于规范行为而非程序缺陷。

解法：短期用自签证书内网验证；正式环境必须上 HTTPS 域名 —— 小程序体验版与正式版也强制要求 HTTPS。

结论：作为已知风险登记 R-31 并给出触发条件；值得注意的是纯 TTS 播报走音频元素不受影响，因此功能是「部分可用」而非「全不可用」。

CHAPTER 04

04

项目完成后的收获

最后回答「做这个项目我得到了什么」—— 技术深度、工程能力、产品思维，以及两项在课程里学不到、只有在真实交付中才会长出来的意识。

01	02	03	04	05
技术深度	工程能力	产品思维	协作规范	风险意识

项目完成后的收获

从「能跑」到「敢上线」的五项成长 · 三端 74 页 · 47,736 行代码 · 121 个接口

五项成长 + 一句总结

这个项目最大的收获不是「把功能写完」，而是理解了**软件质量取决于结构，而不是取决于人的自觉** —— 安全不靠开发者记得过滤，而靠代码的调用关系保证一定过滤；一致性不靠逐页点检，而靠注册表与契约保证不会漏。

01 技术深度：吃透多智能体编排

第一次把业务系统真正拆成「图」：22 个节点、条件边短路、并行分支汇合、子图嵌套、状态在节点间流转。 更深一层的收获是理解了**大模型应该被放在哪里** —— 让它只做擅长的意图识别， 把安全判定与排序计算交回规则与算法。

02 工程能力：质量基线与可复现

后端 319 条单测、Web 端 236 条、小程序端 45 条、43 条端到端用例与 12 条安全红线用例 —— 它们的意义不在于数量，而在于让「改一处会不会弄坏别处」有了确定答案。同时学会让结果**可复现**： 部署写脚本、口径写文档、结论附命令。

03 产品思维：用户敢不敢用

推荐不足时如实说明「宁缺毋滥」而不是用低分菜谱填满列表；被过滤的菜谱提供开关让用户查看原因；营养缺失时标注「未纳入统计」而不是静默按 0 计算。 这些在功能清单上都不算新功能，却决定了用户是否相信产品 —— **可信度本身就是产品力**。

04 协作规范：把约定变成流程

在真实甲乙方协作中第一次体会到流程的价值：需求变更一律走变更单、口头变更无效，文档权限分层、冲突以需求文档为准、术语口径严格区分。 看似繁琐，但它让「上次明明说好了」这类争议不再发生 —— **规范的作用是降低沟通成本**。

05 风险意识：把没做完的写在明面上

项目后期主动整理了 11 条风险（R-23 ~ R-33），包含备案资质周期、程序倒挂、服务器内存压力、语音依赖 HTTPS 等尚未闭环的问题， 并逐条标注概率、影响与应对方式。这一课很重要：**把风险说出来不是暴露短板，而是让项目可控**。

06 一句话总结

这次项目让我真正明白：一个能跑通的 Demo，和一个敢交给用户的产品之间，隔着的正是这些看不见的结构、契约与流程。

下一步计划：补齐备案与签约资质、上 HTTPS 域名、落地语义向量升级与埋点体系，把阶段二小程序推上正式环境。

谢谢

感谢您的耐心聆听 · 请老师指正

PRESENTED BY	PROJECT	DATE
肖裔雄	AI 厨娘 · 智能烹饪助手	2026.10.14

LANGGRAPH MULTI-AGENT · FOOD SAFETY GUARDRAILS · EXPLAINABLE RECOMMENDATION